

A HIRO’s Journey: Hierarchical Learning with Invertible Policies and Blob Goals

Sean Campbell
Robotics
University of Colorado Boulder
sean.campbell-1@colorado.edu

Chris Guthrie
Computer Science
University of Colorado Boulder
guthrie@colorado.edu

Abstract—We explore two modifications to the HIRO hierarchical reinforcement learning algorithm aimed at improving sample efficiency. The first replaces HIRO’s off-policy correction with an invertible-network–based sampling method, while the second adjusts the low-level policy’s reward function through an ellipsoidal goal-weighting scheme. Evaluating both approaches in a simplified Ackermann-steering car U-maze environment, we find that neither improves upon baseline HIRO. However, the invertible-network off-policy correction performs better than using no correction at all, suggesting potential value with better hyperparameter tuning or perhaps more complex environments.

I. INTRODUCTION

While Hierarchical Reinforcement Learning (HRL) encompasses a range of definitions, this work focuses specifically on its value for long-horizon control. By decomposing tasks into policies that operate at distinct temporal scales, HRL methods promise to mitigate the difficulty of long-term credit assignment. We build upon HIRO [1], a two layer framework in which a high-level policy directs a low-level controller via goal specifications.

II. BACKGROUND AND RELATED WORK

A. Options Formalism

What distinguishes hierarchical RL from other methods is a separation of short-timescale (or lower-level) and long-timescale (or higher-level) policies. Lower-level policies are often formalized as *options*, or policies joined to starting and stopping functions. Higher-level policies then select among available options. Options were defined and first theoretically treated in [2].

B. Feudal Reinforcement Learning

A particularly intuitive approach to hierarchical RL is the *feudal* approach [3], in which the higher level agent sets goals for the low-level controller and rewards the low-level controller based on how well the low-level controller achieves those goals. One can think of this approach as options indexed by goals. The basic feudal framework has been explored and adapted to multiple problem settings and learning architectures, including modern deep RL [4].

C. The Option-Critic Architecture

While the options framework provided a theoretical basis for temporal abstraction, many works assume pre-defined sub-policies and/or termination conditions. The Option-Critic architecture [5] extended this by deriving policy gradient theorems for both the option policies and their termination functions. This allows for a learning process where the agent discovers options (i.e. low-level policies) that maximize the expected return of the high-level policy without requiring defined subgoals or low level rewards. In practice, this algorithm may be difficult to use because the learned options can become degenerate; the options may collapse to a single flat policy where the high level always selects the same option or the options selected will switch every timestep.

D. FeUdal Networks

Building on the original feudal reinforcement learning idea, FeUdal Networks (FuNs) [4] use a manager that sets abstract goals in a latent state space and a worker that attempts to achieve them. Unlike HIRO, which defines goals in the state space (or a subset), FuNs rely on directional cosine similarity in the latent space to reward the Worker.

E. The HIRO Formulation

HIRO [1] is a two-level hierarchical reinforcement learning framework designed to improve sample efficiency in long-horizon continuous-control tasks. The high-level (manager) policy operates at a coarse temporal scale and outputs goal vectors in the original state space every c steps that specify desired state transitions. These are directional targets rather than explicit actions. The manager receives only the environment’s external reward signal, while the low-level (worker) policy is trained solely on an intrinsic reward defined with respect to the manager’s goals.

So broadly speaking, HIRO belongs to the class of feudal RL approaches. However, whereas Feudal Networks [4] learn a latent goal representation, HIRO defines goals directly in the state space, which simplifies training and improves stability. The worker is trained to reach the goal state $s_t + g_t$ by maximizing a shaped reward given by

$$r(s_t, g_t, a_t, s_{t+1}) = -\|s_t + g_t - s_{t+1}\|_2, \quad (1)$$

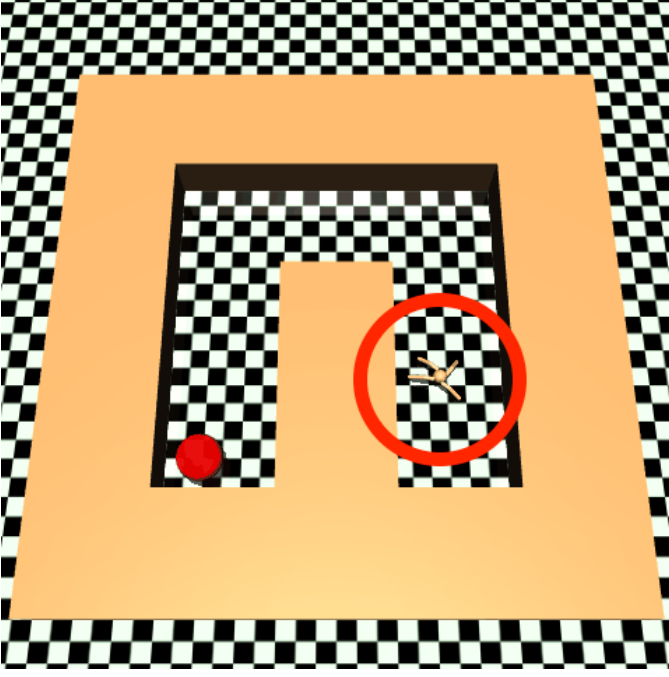


Fig. 1. Ant Maze (Ant Circled). Image from Gymnasium Website. [6]

which penalizes the Euclidean distance between the intended goal and the resulting transition. This separation of reward channels and temporal scales allows the worker to learn short-horizon control skills while the manager focuses on long-horizon planning.

As the worker moves through the environment, the manager’s goal is updated to keep track of the remaining displacement. Specifically, if the worker moves from s_t to s_{t+1} , the goal transitions according to

$$g_{t+1} = g_t - (s_{t+1} - s_t), \quad (2)$$

This provides the worker with an updated direction from the original goal set by the manager.

The original HIRO paper empirically validated the algorithm on a Mujoco “Ant Maze,” in which a simulated quadruped robot needs to navigate around a wall. See Fig. 1. They found that it significantly outperformed TD3 (a non-hierarchical algorithm) on both this task and several others.

For “Ant Maze”, the higher-level controller sets a goal position for the ant in the maze and an independent lower-level controller gets c steps to attempt to move in the goal direction by applying torque to the ant’s actuators. In the original HIRO paper, the goal direction only incorporates the (x, y) coordinates of the ant in the maze and not the ant’s internal state. In our adaptation, however, we will consider goals which also consider the ant’s internal state. As the ant moves towards or away from the goal direction, the direction is adjusted so that the ultimate goal position remains unchanged until the c -step subepisode is complete and the higher-level controller receives the cumulative environment reward from the past c steps and decides on a new goal.

Maze environments are well-suited to hierarchical learning approaches because they have a natural temporal hierarchy. Lower-level actions need to control the robot to move effectively in a particular direction, while higher-level planning is also necessary to navigate successfully around walls.

F. Problem Formulation

Our objective is to improve HIRO’s sample efficiency on a simplified toy environment that we created, the Car Maze. While the original HIRO paper evaluates performance on the Ant U-Maze and other high-dimensional continuous-control tasks, such environments are computationally demanding and take over 6 hours to train in our setup. Due to resource and time constraints, we focus on the lower-dimensional Car Maze setting, allowing for more rapid iteration and analysis.

G. Car Maze Details

We adapt the PointMaze U-maze environment to a car navigation task. The underlying maze layout is the same as the Ant U-maze, but the agent is now a planar point with a low-dimensional state which follows a simplified model of standard (Ackerman) car steering.

1) *States, Observations, and Transitions:* The car moves in the (x, y) -plane with heading angle θ_t and scalar speed v_t . We define the car state and the agent’s observation as

$$s_t = (x_t, y_t, v_{x,t}, v_{y,t}, \cos \theta_t, \sin \theta_t),$$

where

$$v_{x,t} = v_t \cos \theta_t, \quad v_{y,t} = v_t \sin \theta_t.$$

In our implementation, we start from the original MuJoCo observation of the PointMaze environment and augment it to follow the kinematic bicycle model.

2) *Maze Layout, Environment Initialization:* We use the discrete U-maze topology from the original Ant PointMaze task, but replace the point/ant agent with the car described above. The maze is specified as a 5×5 grid with walls, a single goal cell “g”, and one or more start cells “r”. Each cell is a 0.5×0.5 m region in which either the goal or car can be placed in the center of with some noise. During training, we randomize over multiple start cells to improve robustness, using

$$\text{training_map} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & g & r & r & 1 \\ 1 & 1 & 1 & r & 1 \\ 1 & r & r & r & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix},$$

while evaluation uses a single fixed start cell

$$\text{eval_map} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & g & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 & 1 \\ 1 & r & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 \end{bmatrix}.$$

Here, entries equal to 1 correspond to walls, entries equal to 0 are free space, “r” are possible start locations, and “g” is the goal. The exact position of the start and the goal within each cell is randomized.

3) *Rewards and Termination*: The reward at each step is the negative Euclidean distance between the agent position and goal positions:

$$r_t = -\|\text{achieved_position}_t - \text{goal}\|_2^2.$$

An episode terminates when the achieved position has Euclidean distance less than 0.5 from the goal.

The discount factor we use is $\gamma = 0.99$.

III. SOLUTION APPROACH

A. Baseline

We aim to follow the HIRO framework for hierarchical RL, where a high-level controller proposes subgoals g_t every c steps and a low-level controller outputs primitive actions a_t conditioned on the current state s_t and subgoal g_t .

We derived our core HIRO implementation from a clone of a public repository [7], which aims to faithfully replicate the original paper [1] in Pytorch.

1) *Architecture*: Both the higher-level and lower-level controllers are implemented as TD3 [8] agents, using Pytorch directly (so no intermediate TD3 library). The actor and critics for each agent are implemented as MLPs with two hidden layers of width 300 each, with ReLU activations. Except for necessary exceptions due to our algorithmic changes, we use the same hyperparameters as in the original cloned repository.

2) *Off-Policy Corrections*: Consistent with the original HIRO framework, our implementation stores experience for both controllers in an un-prioritized replay buffer. However, because the lower-level policy evolves during training, the goals originally selected by the high-level controller may no longer produce the recorded trajectories. To address this non-stationarity, HIRO applies off-policy corrections when sampling high-level transitions. Specifically, it generates a set of candidate goals (the original goal, the realized state displacement by the worker, and 8 samples from a Gaussian centered on that displacement). HIRO then evaluates the log-probability of the recorded lower-level actions given each candidate goal using the current lower-level policy. The candidate maximizing this probability is selected as the relabeled goal for the high-level update.

3) *Our Customizations*: We left the core logic of the algorithm untouched, although we had to refactor and generalize things in a few places. We had to decouple the algorithm implementation from the shape of the environment to accommodate our test environment. We also needed to do some miscellaneous upgrades to modern Gymnasium APIs. Finally, we factored out the structure of the intermediate subgoals to allow for swapping in our own custom subgoal implementations and off-policy corrections.

B. Invertible Low-Level Policy and Off-Policy Correction

In contrast to the original HIRO off-policy correction which is based on sampling candidate subgoals near the observed state transition, we parameterize the low-level policy as an

invertible neural network and use its inverse to perform off-policy correction. To ensure that these inverses are reasonable off-policy corrections, we add additional losses during training. Otherwise, the original HIRO algorithm remains unchanged.

1) *Invertible Neural Network Architecture*: The low-level policy π_{low} is implemented as an *invertible neural network* (see [9]–[11]) that maps the concatenated input (s_t, g_t) to an output vector whose first coordinates define the environment action, the next set of coordinates define a reconstructed state, and the final coordinates “latent” outputs. The network is constructed so that every layer has an explicit inverse, allowing us to recover (s_t, g_t) from the output.

Given the input

$$x = [s_t, g_t] \in \mathbb{R}^{d_s + d_g},$$

the model applies a sequence of L invertible “flow steps”

$$x^{(0)} = x, \quad x^{(\ell+1)} = f^{(\ell)}(x^{(\ell)}), \quad \ell = 0, \dots, L-1.$$

Each step combines two analytically invertible transformations:

- 1) **Affine coupling transformation**. A binary mask $m \in \{0, 1\}^{d_s + d_g}$ partitions the vector into $x_{\text{id}} = x_{m=1}$ (passed through unchanged) and $x_{\text{tr}} = x_{m=0}$ (transformed). An MLP predicts scale and shift parameters $(s(x_{\text{id}}), t(x_{\text{id}}))$, and we update

$$x'_{\text{tr}} = x_{\text{tr}} \odot \exp(\tanh s) + t.$$

Because the identity part is unchanged, the inverse is closed-form:

$$x_{\text{tr}} = (x'_{\text{tr}} - t) \odot \exp(-\tanh s).$$

- 2) **Invertible linear transformation**. The entire vector is multiplied by an orthogonal matrix W obtained from a QR decomposition at initialization:

$$x' = xW.$$

Since W is orthogonal, the inverse is $W^\top$.

Why do we combine these two invertible transformations?

The reason is that the affine coupling transformation has limited expressivity by itself since half of the inputs are passed through unchanged; however, if we mix outputs via an invertible linear transformation after each affine layer, we can greatly increase the types of functions that can be learned. After all L blocks, we obtain the output

$$x^{(L)} = [h^{\text{act}}, h^{\text{state}}, h^{\text{lat}}],$$

where:

- the first d_a coordinates h^{act} define the unscaled action,
- the next d_s coordinates correspond to a reconstructed state $\tilde{s}_t$,
- the remaining $d_g - d_a$ coordinates form a latent vector z_t .

At the final “flow step”, we bound the action output by applying tanh and a scale param only to the action coordinates:

$$a_t = \tanh(h^{\text{act}}) \odot \text{scale}.$$

The remaining dimensions are returned unchanged.

2) *Inverse Mapping*: The low level policy is implemented as an invertible network of 4 layers of “flow steps”, each with 300 hidden units. To invert the network, we first unsquash the action using an elementwise arctanh, reconstruct the full output vector, and then apply all invertible steps in reverse order:

$$(\tilde{s}_t, g_t) = \pi_{\text{low}}^{-1}(a_t, s_t, z_t).$$

Thus, given an action and state pair, we can generate a latent and invert to get a reconstructed state and plausible goal.

3) *Additional Losses*: To ensure that the low level network produces suitable inverses, we add 4 additional losses that are added to the original low level actor loss.

- 1) **Cycle consistent action reconstruction loss**: Given a transition (a_t, s_t, g_t) from the replay buffer, we construct the tuple

$$(a_t, s_t, N(0, 1)),$$

using the true state s_t , previously produced action a_t and a sampled latent. Inverting through the network yields an inferred subgoal (and a reconstructed state which we ignore):

$$(\hat{s}_t, \hat{g}_t) = \pi_{\text{low}, \phi}^{-1}(a_t, s_t, N(0, 1)).$$

Pushing the inferred goal back through the actor gives a reconstructed action,

$$\hat{a}_t = \pi_{\text{low}, \phi}(s_t, \hat{g}_t)_{\text{action}},$$

and we define the action-reconstruction loss

$$\mathcal{L}_{\text{act}} = \mathbb{E}[\|a_t - \hat{a}_t\|_2^2].$$

This loss encourages the network to produce goals that are on policy for the high level controller.

- 2) **Original goal reconstruction loss**: To stabilize training, we add a goal-reconstruction loss:

$$\mathcal{L}_{\text{goal}} = \mathbb{E}[\|\hat{g}_t - g_t\|_2^2].$$

While theoretically this loss is not needed, it helps prevent the network from generating inverses that are significantly different from goals that have been seen before.

- 3) **State and latent losses**: We add losses to ensure that the reconstructed state matches the input state and that the latent variables look like unit Gaussians.

$$\mathcal{L}_{\text{state}} = \mathbb{E}[\|\tilde{s}_t - s_t\|_2^2], \quad (3)$$

$$\mathcal{L}_{\text{latent}} = \mathbb{E}[\|\mu_{\text{batch}}\|_2^2 + \|\sigma_{\text{batch}} - 1\|_2^2] \quad (4)$$

$$= \mathbb{E}\left[\frac{1}{d_z} \sum_{i=1}^{d_z} \mu_{\text{batch}, i}^2 + \frac{1}{d_z} \sum_{i=1}^{d_z} (\sigma_{\text{batch}, i} - 1)^2\right] \quad (5)$$

where d_z is the latent dimension, μ_{batch} and σ_{batch} are the batch mean and standard deviation. This encourages z_t to match a standard normal distribution that we can then sample from to generate multiple inferred subgoals per (a_t, s_t) tuple.

- 4) *RL Objective*: The actor’s RL loss follows the standard TD3 actor loss that is minimized when the Q value is maximized:

$$\mathcal{L}_{\text{RL}} = -\mathbb{E}[Q_{\psi}(s_t, g_t, a_t)].$$

a) *Total Low Actor Loss*: The total loss for the low level policy is a weighted sum of the individual losses listed above. During our limited hyperparameter sweeps, we found that weights of 1 performed reasonably well. The high level actor loss does not change and we treat this low level actor loss as an augmented version of the original TD3 low level actor loss and train it at the same cadence.

$$\mathcal{L}_{\text{actor}} = \mathcal{L}_{\text{RL}} + \lambda_{\text{state}} \mathcal{L}_{\text{state}} + \lambda_{\text{latent}} \mathcal{L}_{\text{latent}} + \lambda_{\text{goal}} \mathcal{L}_{\text{goal}} + \lambda_{\text{act}} \mathcal{L}_{\text{act}}.$$

5) *Off-policy correction via the invertible model*: For high-level off-policy correction, we replace the original HIRO sampling procedure with candidates obtained from the inverse of the invertible low-level policy. Given a sequence of low-level transitions $\{(s_{t+k}, a_{t+k})\}_{k=0}^{K-1}$, we construct the tuples

$$(a_{t+k}, s_{t+k}, \mathbf{0}),$$

where the latent component is set to zero. While it is possible to sample latents instead of setting them to 0, we saw slightly reduced performance when we tried this variation. Inverting these through the invertible low level network yields reconstructed states and plausible subgoals:

$$(\hat{s}_{t+k}, \hat{g}_{t+k}) = \pi_{\text{low}, \phi}^{-1}(a_{t+k}, s_{t+k}, \mathbf{0})$$

These inverted subgoals are goals that have been shifted using Equation (2) and we must transform them back to the original goal that the manager provided at the beginning of the sequence. Namely, we must solve for $\tilde{g}_t$ using $\tilde{g}_{t+k}$, our inverted subgoal:

$$\tilde{g}_{t+k} + (s_{t+k} - s_t) = \tilde{g}_t$$

This gives us a candidate set

$$\{\tilde{g}^{(i)}\}_{i=1}^K$$

Each candidate is scored according to how well it explains the observed low-level actions under the current policy. This scoring procedure is identical to the original HIRO algorithm method.

$$\ell^{(i)} = -\frac{1}{2} \sum_{k=0}^{K-1} \|\pi_{\text{low}, \phi}(s_{t+k}, \tilde{g}^{(i)})_{\text{action}} - a_{t+k}\|_2^2$$

The corrected subgoal is chosen as

$$g_t^{\text{OPC}} = \arg \max_{\tilde{g}^{(i)}} \ell^{(i)}$$

This invertible-model off-policy correction replaces the heuristic candidate sampling used in HIRO.

C. Blob (Ellipsoid) Goals

This adjustment to HIRO hones in on one main problem with how the manager network sets goals for the worker network: because it uses the ℓ^2 norm (eqn. 1), it treats every dimension of the state space as equally important.

The idea is that instead of using the ℓ^2 norm, we can use an ellipsoidal norm whose parameters are defined by the manager agent. So, instead of the manager outputting a single goal direction $g_t \in \mathbb{R}^S$, the manager will output both a goal state $u_t \in \mathbb{R}^S$ and (some representation of) a positive-definite matrix $A_t \in \mathbf{S}_S^{++}$ of size $S \times S$, which induces a norm $\|x\|_{A_t} = \sqrt{x^T A_t x}$.

In that case, the worker agent’s reward will be:

$$\begin{aligned} r(s_t, u_t, A_t, s_{t+1}) &= -\|s_{t+1} - u_t\|_{A_t} \\ &= -\sqrt{(s_{t+1} - u_t)^T A_t (s_{t+1} - u_t)} \end{aligned}$$

Using an ellipsoid allows the manager agent to learn that some parts of the state space are more or less relevant and at different scales.

1) *The General and Diagonal Formulations:* In this fully general formulation, the ellipsoid defined by A_t can be oriented in any direction in state space, so the axes of A_t need not be aligned with the dimensions of the state. This theoretically would allow the manager agent to represent tradeoff-based goals to the worker agent by changing the axes of the ellipsoid.

However, we only test ellipsoidal transformations which are aligned to the original axes of the state space – so in practice, we only consider diagonal A_t .

2) *Adjusting Historical Samples to a Perfect Policy:*

Whereas HIRO attempts to adjust historical samples to the current worker policy, the ellipsoid approach adjusts historical samples to a “perfect” worker policy which can achieve optimal loss on any goal. It aims to relabel history samples s, a, s', r, u, A with some u^*, A^*, r^* such that $r^* = -\|s'\|_{A^*, u^*} \approx 0$.

This is in the spirit of Hindsight Experience Replay [12], certainly in the spirit of Feudal Networks [4]. It’s also a simpler approach than what the authors of HIRO actually do. The authors of HIRO claim to have tried a version of this approach in their paper (in which they relabeled the goal to the actual achieved state plus some unit-variance noise), but found that it “did not outperform” their more complex approach. The simplicity may come at a cost though, since this off-policy correction only considers the goal and the final state and does not try to match the entire episode trajectory.

The most straightforward solution here would be to simply relabel $u^* = s'$. However, this does not accommodate the fact that the “perfect” worker policy may still not be able to exactly achieve the given goal, due to the stochasticity of the environment. The manager agent needs to be able to plan around this.

Instead, we choose to relabel $A \rightarrow A^*$, so that under A^* , the manager retrospectively claims to care very little, but not zero, about the actual error between s' and u (Say, the manager claims that the loss of s' with respect to u is less than 1). The

interpretation then would be that the “perfect” worker is able to consistently put the environment into a state which lies inside the unit ellipsoid defined by A .

In this approach, we do not relabel the goal state at all. However, one could relabel the goal state by adding noise generated from A , rather than expanding A to include the goal state. It would be interesting to see how this change could affect the behavior and performance of the ellipsoid goals.

3) *Adjusting the Ellipsoid:* In full generality, relabeling $A \rightarrow A^*$ so that the distance between goal and final state is at most 1, is an interesting and tractable linear algebra problem, equivalent to dilating the ellipsoid defined by A in the direction between goal and state. When we know that A is diagonal, however, the interpretation becomes less clear. It also becomes less obvious how to decide which axes of the ellipsoid to dilate.

We did not exactly resolve these questions and resorted instead to a simple heuristic algorithm. We iteratively evaluate the differences between goal and final state, with each dimension weighted by the size of the ellipsoid along that dimension, and slightly increase those dimensions which show the largest difference, until the overall ellipsoidal distance is less than 1.

4) *Implementation Details:* We limit the radii of the ellipsoid to be at most $e_{\max}$ along each dimension. We experimented with different values of $e_{\max}$, typically 1, 2, 4, 8, and 16. Setting $e_{\max} = 1$ should be functionally equivalent to running HIRO with no off-policy corrections.

We represent the new subgoals to the worker as a concatenation of the goal direction and an array of the log of the radii for each dimension.

$$s_{\text{worker}} = (s_{\text{env}}, g, e)$$

for $s_{\text{env}}, g, e \in \mathbb{R}^{D_s}$, where D_s is the dimension of the state space.

The worker is then rewarded based on its clipped ellipsoid distance to the goal:

$$d(s_t, s_{t+1}, g_t, e) = \|(s_t + g - s_{t+1}) / \exp(\min(e, 0))\|_2$$

where the operations $\exp$, $\min$, and $/$ are elementwise. We use the clipping $\min(e, 0)$ to preserve stability by keeping the worker’s rewards bounded by the diameter of the state space.

IV. RESULTS

A. Methodology

We ran each algorithm 10 times in the Car Maze environment with different seeds and plotted its mean reward and success rate. In the plots below, the solid lines are mean performance with shaded areas representing standard deviations. Success rate is the fraction of time that the car is able to navigate from one end of the maze to the other. Each episode is approximately 200-300 steps.

B. Car Maze: Invertible Nets

We were able to get our invertible network approach to perform similarly to the original HIRO algorithm and better than HIRO with no off policy correction (i.e. just use the

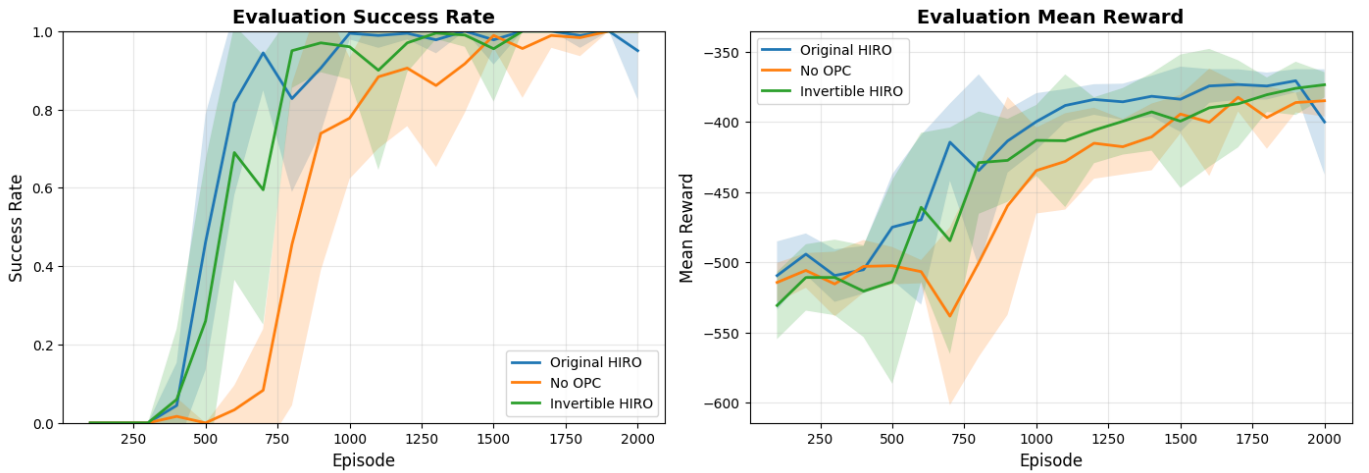


Fig. 2. Invertible neural network (ours) mean success and reward over 10 runs compared to HIRO and HIRO with no off policy correction (No OPC).

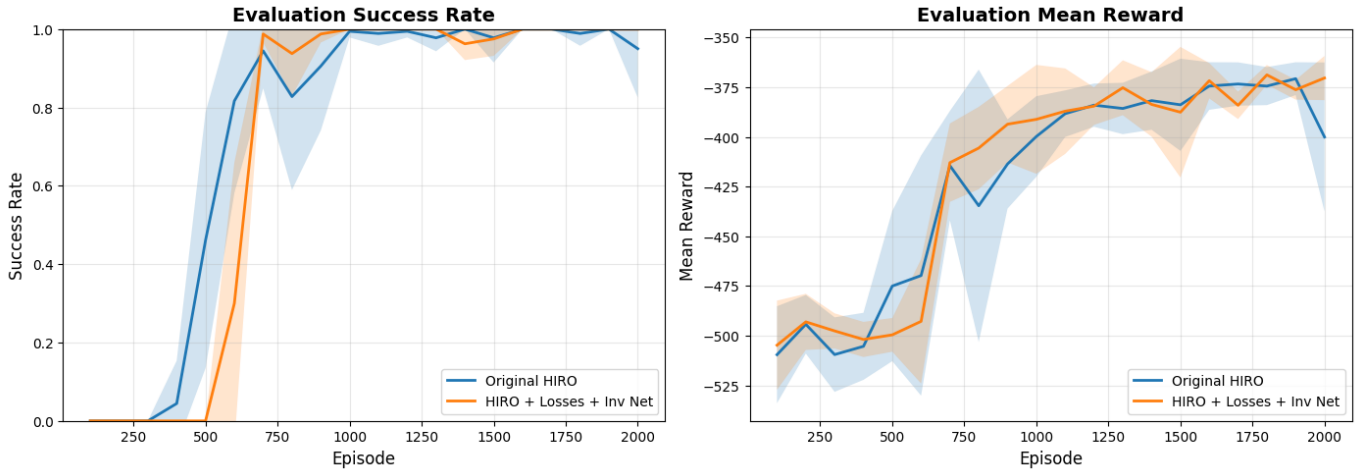


Fig. 3. Original HIRO algorithm compared to the HIRO algorithm using an invertible net and invertible losses (but original HIRO off-policy correction)

original goal in the replay buffer) in terms of final performance and sample efficiency (see Fig. 2). We did try a number of seemingly obvious tweaks which did not improve performance though. These included pretraining the invertible networks, sampling more inverted goals by generating many more latents, and using different loss weights.

Are invertible nets not expressive enough? To test that the invertible net architecture was not limiting the performance of our algorithm, we ran HIRO with its normal off policy correction but with the low level actor replaced with an invertible network (see Fig. 3). We found that the performance did not change even when adding the invertible net losses.

C. Car Maze: Ellipsoid Goals

The initial implementation of the ellipsoid goals on top of our base HIRO implementation (with no hyperparameter tuning) had somewhat dispiriting results:

The results confirmed that setting $e_{\max} = 1$ behaves indistinguishably from HIRO with no off-policy correction. The

subject failure of the radius 16 experiment appeared to be due to instability during training.

1) *Normalized Subgoals*: We hypothesized that the instability in the $e_{\max} = 16$ experiment was an issue with the larger size of the goal inputs (up to 16) into the worker controller. So, we tried normalizing the size of the goals to always be one, and scale up when calculating the worker’s reward. This made the ellipsoid experiments perform even worse (see Fig. 5), but did resolve the training instability.

2) *Potential-Based Rewards*: Since we thought we would be needing to retune the algorithm anyway, we adjusted the reward scheme to be potential-based since that kind of reward shaping is more theoretically grounded [13] and aligned to the ultimate goal of achieving success in the maze. This meant adjusting both the base environment and the low-level worker rewards to be of the form:

$$r(g_t, s_t, s_{t+1}) = \|g_t\| - \gamma \|s_t + g_t - s_{t+1}\|$$

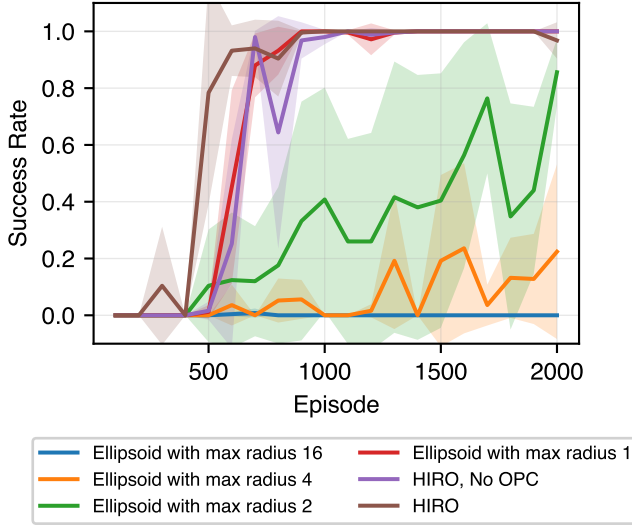


Fig. 4. Success of Ellipsoid Method For Varying $e_{\max}$

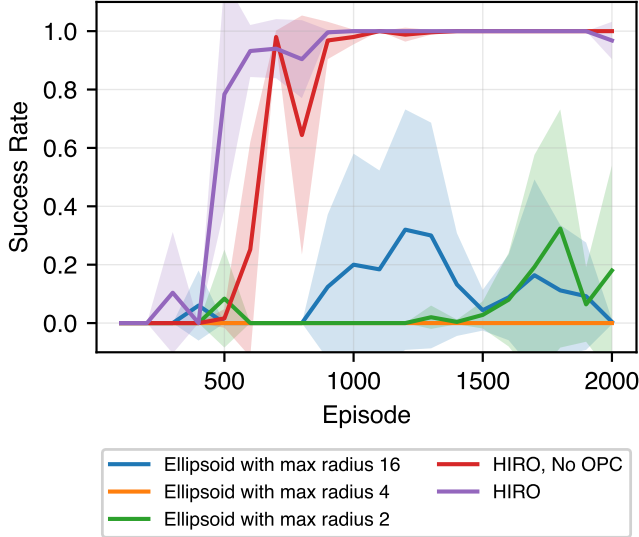


Fig. 5. Success Rate of Ellipsoid Method with Normalized Subgoals

and adding a large reward of 200 to the environment when an episode terminated with success.

Changing the reward structure may have been an unwise idea, as it led to even worse performance, with zero successes before hyperparameter tuning and no sign of learning over time. See Fig. 6. It also proved to be very difficult to tune. (That could be a skill issue though).

V. CONCLUSION

Hierarchical reinforcement learning shows some promise in learning hierarchical tasks like robot mazes. However, HIRO in particular, seems quite sensitive to hyperparameters and reward schemes. We would suggest that the limiting factor in implementing HIRO successfully is far more likely to be

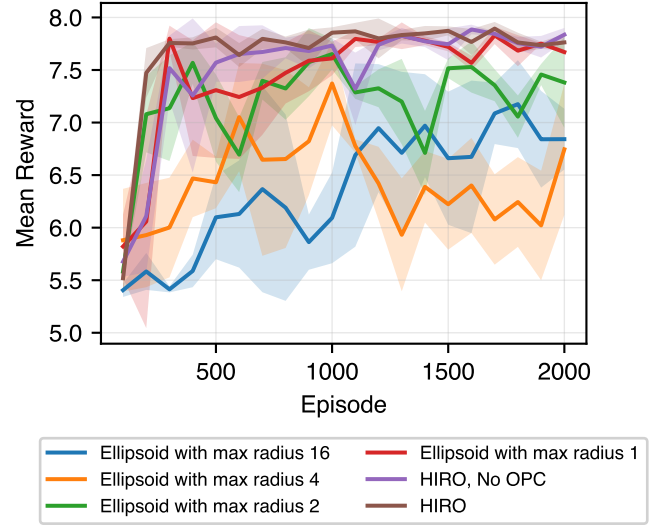


Fig. 6. (Lack of) Learning with Potential-Based Rewards. Note that even the HIRO methods failed to succeed on the task.

the difficulty in tuning it for a particular environment than it is to be the fundamental inefficiencies of the algorithm.

A. Environment selection

We selected the Car Maze as our evaluation environment because we wished to iterate quickly on our ideas. However, a single run takes 30-60 minutes and we needed 5-10 runs per experimental configuration to confidently draw any conclusions. Thus, our iteration speed was limited to roughly 2 experiments per day which was prohibitively slow given this project's timeline.

Furthermore, the Car Maze environment was not difficult enough to strongly differentiate between HIRO, our algorithms, and HIRO with no off policy correction. While we did check that TD3 struggled with the Car Maze (though it does eventually solve it), we should have picked an environment that was a bit more difficult so that we could more clearly distinguish algorithmic improvements.

B. Ellipsoid Goals

The ellipsoid subgoal method is likely not a good fit for the Car Maze problem. It's very possible that the observation space of the Car Maze environment is simply too small for whatever benefits the ellipsoid method might have to outweigh the overhead of needing to learn a larger action space (i.e. the subgoal space).

C. The Reward Shaping Side Quest

The failure of potential-based rewards, even with some hyperparameter tuning, is also intriguing. It suggests that hierarchical approaches are not a panacea for long-horizon planning and exploration, and careful design of rewards, exploration strategies, and planning remains necessary to achieve good performance in general. However, a more thorough

hyperparameter search here would be necessary in order to make these claims confidently.

D. Future Work

1) *Ellipsoid Goals*: A more extensive hyperparameter search would be a good place to start to continue investigation into this method. The error tolerance for the relabeling of A is a hyperparameter which we did not experiment with. It could also be worth testing the method out on an environment with a larger state space.

CONTRIBUTIONS

Both Sean and Chris worked on the initial environment setup. Sean focused exclusively on the invertible neural network approach while Chris worked on the ellipsoid goals approach.

PERMISSIONS

The authors grant permission for this report to be posted publicly.

ALGORITHM IMPLEMENTATION

We built off the HIRO pytorch library; however, we had to make surprisingly extensive modifications to accommodate our variations of the HIRO algorithm. See Section III-A.

REFERENCES

- [1] O. Nachum, S. Gu, H. Lee, and S. Levine, “Data-efficient hierarchical reinforcement learning,” 2018. [Online]. Available: <https://arxiv.org/abs/1805.08296>
- [2] R. S. Sutton, D. Precup, and S. Singh, “Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning,” *Artificial intelligence*, vol. 112, no. 1-2, pp. 181–211, 1999.
- [3] P. Dayan and G. E. Hinton, “Feudal reinforcement learning,” *Advances in neural information processing systems*, vol. 5, 1992.
- [4] A. S. Vezhnevets, S. Osindero, T. Schaul, N. Heess, M. Jaderberg, D. Silver, and K. Kavukcuoglu, “Feudal networks for hierarchical reinforcement learning,” *CoRR*, vol. abs/1703.01161, 2017. [Online]. Available: <http://arxiv.org/abs/1703.01161>
- [5] P.-L. Bacon, J. Harb, and D. Precup, “The option-critic architecture,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 31, no. 1, 2017. [Online]. Available: <https://arxiv.org/abs/1609.05140>
- [6] Farama Foundation, “Ant maze - gymnasium-robotics documentation,” 2025. [Online]. Available: https://robotics.farama.org/envs/maze/ant_maze/
- [7] K. Watanabe, “hiro_pytorch,” 2019, accessed 2025-12-03. [Online]. Available: https://github.com/watakandai/hiro_pytorch
- [8] S. Fujimoto, H. Hoof, and D. Meger, “Addressing function approximation error in actor-critic methods,” in *International conference on machine learning*. PMLR, 2018, pp. 1587–1596.
- [9] L. Dinh, D. Krueger, and Y. Bengio, “Nice: Non-linear independent components estimation,” 2014. [Online]. Available: <https://arxiv.org/abs/1410.8516>
- [10] L. Dinh, J. Sohl-Dickstein, and S. Bengio, “Density estimation using real nvp,” 2016. [Online]. Available: <https://arxiv.org/abs/1605.08803>
- [11] D. P. Kingma and P. Dhariwal, “Glow: Generative flow with invertible 1x1 convolutions,” in *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*, 2018. [Online]. Available: <https://papers.nips.cc/paper/8224-glow-generative-flow-with-invertible-1x1-convolutions>
- [12] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, P. Abbeel, and W. Zaremba, “Hindsight experience replay,” *CoRR*, vol. abs/1707.01495, 2017. [Online]. Available: <http://arxiv.org/abs/1707.01495>
- [13] A. Y. Ng, D. Harada, and S. Russell, “Policy invariance under reward transformations: Theory and application to reward shaping,” in *ICML*, vol. 99. Citeseer, 1999, pp. 278–287.